

Pantallas en Flutter y Agentes en Consola

Maquetación Declarativa, Autonomía Agéntica y
Contratos con AGENTS.md

Seminario de I+D+I — Universidad Icesi

01. Layout y Composición

De la estructuración arquitectónica de pantallas al control preciso de cajas, proporciones y desbordes en Flutter.

Arquitectura de Vistas: Screen frente a Page

Separación de Responsabilidades: La `Screen` gestiona la ruta en `MaterialApp` y devuelve el único `Scaffold`. La `Page` vive dentro del `body` como lienzo de contenido y tiene estrictamente prohibido devolver un `Scaffold`.

✓ Regla de Oro: Arquitectura Limpia

Screen (Un solo Scaffold)

AppBar (Marco Global & TabBar)

body: Page (Sin Scaffold)

SafeArea > Column

Widgets Propios (Cards, Badges)

Expanded / Scrollable View

BottomNavigationBar / FAB

1 Pantalla = 1 Scaffold • Cero conflictos

✗ Antipatrón: Scaffolds Anidados

Scaffold Externo

AppBar Exterior

Page > Scaffold (¡ERROR!)

Doble AppBar (Desperdicio de espacio)

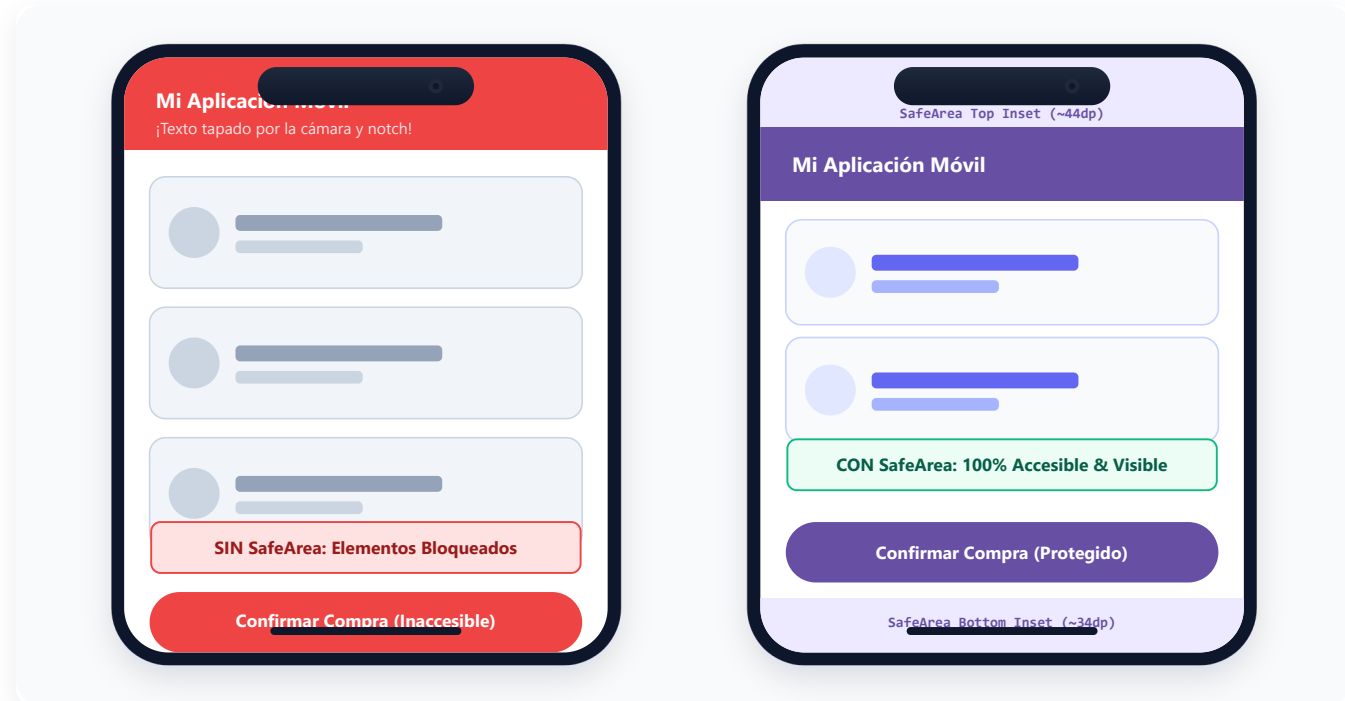
● **Bug de Teclado:**
Doble `resizeToAvoidBottomInset` tapa inputs

● **Conflicto con Snackbar:**
Se oculta tras la barra o rompe el contexto

Nunca devuelvas un Scaffold dentro de una Page

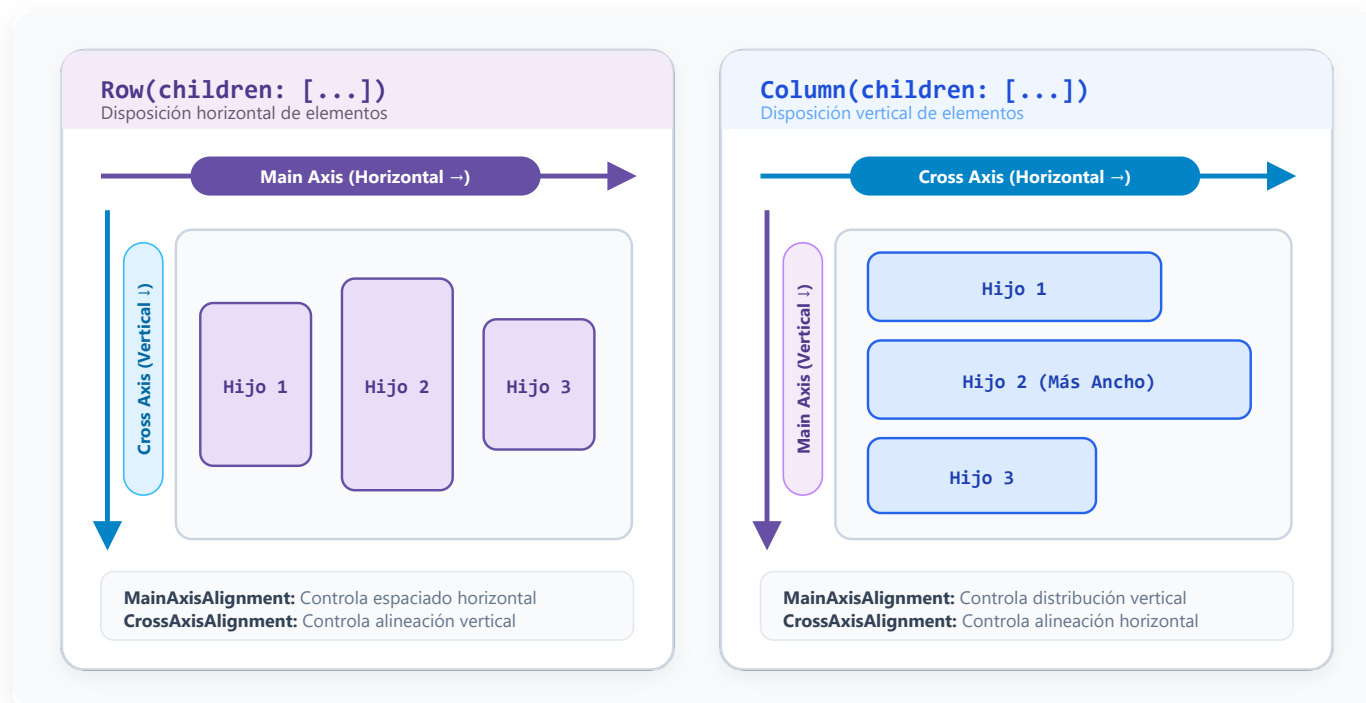
Andamiaje y Hardware Insets: Scaffold y SafeArea

Protección de Interfaz: `SafeArea` calcula dinámicamente los insets del dispositivo físico (cámara frontal tipo notch, esquinas redondeadas y barra de gestos inferior) garantizando que ningún botón quede tapado.



Distribución Multieje: Column y Row

Ejes Relativos: `MainAxisAlignment` rige el flujo natural del widget (vertical en Column, horizontal en Row), mientras que `CrossAxisAlignment` controla la alineación perpendicular respecto al ancho disponible.

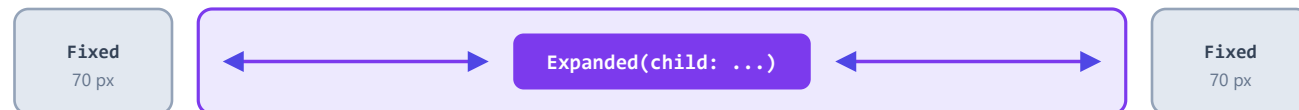


Proporcionalidad y Flexibilidad: Expanded y Spacer

Control del Espacio Restante: El widget `Expanded` fuerza a su hijo a ocupar todo el espacio disponible. Mediante el factor `flex` se define la proporción matemática de reparto entre elementos hermanos.

Caso 1: Cajas de Ancho Fijo + Expanded Central

Row(children: [...])



El widget `Expanded` absorbe todo el ancho residual disponible en el eje principal, adaptándose a cualquier tamaño de pantalla sin desbordar.

Caso 2: Distribución Proporcional con Factor flex (33.3% / 66.6%)

flex: 1 vs flex: 2



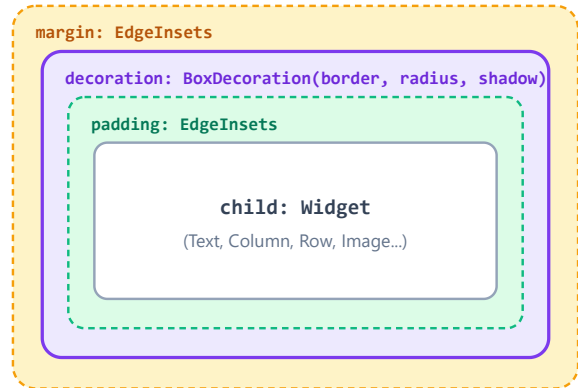
El parámetro `flex` divide el espacio total en cuotas proporcionales ($1 + 2 = 3$ partes). Elimina por completo los cálculos manuales con `MediaQuery`.

Modelo de Cajas: Container y BoxDecoration

Anatomía Visual de la Caja: `Container` compone márgenes exteriores, bordes perimetrales, radio de curvatura (`borderRadius`), relleno interno (`padding`) y fondo cromático alineado a Material Design 3.

Container() Box Model

Capas concéntricas de dimensiones y estilo



Si solo necesitas separación interior, usa **Padding** (más liviano).

¡Peligro! Regla de BoxDecoration

Error de aserción en tiempo de ejecución

✗ Código que detona error fatal

```
Container(  
  color: Colors.blue, // <-- ¡ERROR!  
  decoration: BoxDecoration(...),  
)
```

✓ Solución correcta (color adentro)

```
Container(  
  decoration: BoxDecoration(  
    color: Colors.blue, // ✓ Correcto  
    borderRadius: BorderRadius.circular(16),  
  ),  
)
```

Control de Desborde: RenderFlex Overflow

Resolución de Desbordes: Cuando una `Column` estática supera la altura física de pantalla aparece el rayado amarillo y negro de RenderFlex. Envolver la jerarquía en `SingleChildScrollView` activa el scroll reactivo.

El Problema: RenderFlex Overflow

`Column()` sin capacidad de desplazamiento



La columna rígida no calcula scroll por defecto

La Solución: SingleChildScrollView

`SingleChildScrollView(child: Column(...))`



Scroll vertical suave y adaptabilidad a cualquier celular

02. De Asistentes a Agentes

El salto operativo desde interfaces pasivas de chat hacia agentes autónomos integrados en el sistema operativo.

¿Qué es un Asistente de IA?

El Paradigma del Asistente: Un modelo de lenguaje interactivo que opera como *consejero pasivo*: recibe un prompt textual y genera una sugerencia o respuesta, pero **no** tiene acceso al sistema operativo ni a la terminal.

Arquitectura de un Asistente Tradicional de IA

Modelo de sugerencia reactiva de texto desacoplado del sistema operativo (ChatGPT / Copilot)



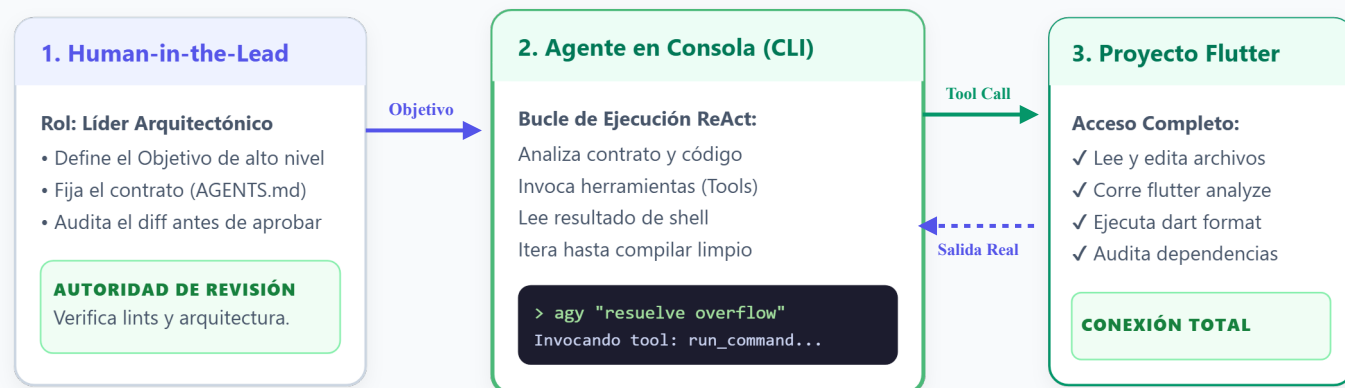
Resumen: Un asistente es un **consejero pasivo**. Genera texto pero carece de manos para interactuar con tu entorno de desarrollo.

¿Qué es un Agente de Consola?

El Paradigma del Agente: Una entidad autónoma que combina un LLM con **acceso a herramientas nativas de tu máquina** (lectura de archivos, ejecución de comandos y parches diff) dentro de un bucle que se auto-corrige hasta cumplir la meta.

Arquitectura de un Agente de Consola (Bucle Autónomo)

Entidad autónoma con acceso directo a herramientas del sistema operativo y auto-corrección



Resumen: Un agente es un **colaborador con autonomía y herramientas**. No solo propone código, sino que lo prueba, lo audita y lo refactoriza en tu propia máquina.

Diferencias Clave: Asistente frente a Agente

Matriz de Capacidades: La diferencia no radica únicamente en la potencia del modelo subyacente, sino en su **grado de agencia, autonomía operativa y conexión con el entorno** de desarrollo.

Dimensión	Asistente Tradicional (Copilot / Chatbot)	Agente en Consola (agy / opencode)
Modo Operativo	Pasivo: Espera preguntas y solo devuelve cadenas de texto.	Activo: Ejecuta secuencias de pasos autónomos para cumplir la meta.
Entorno	Aislado en navegador o en un panel lateral del IDE.	Integrado en la terminal con acceso al repositorio local y archivos.
Herramientas	Ninguna: no ejecuta comandos ni navega el árbol de código.	Nativas: lee archivos, inspecciona el AST y ejecuta scripts en shell.
Verificación	Ciega: desconoce si el código compila o tiene advertencias.	Verificada: ejecuta <code>flutter analyze</code> y auto-corrige sus propios lints.



Sin Shell



Fricción



Amnesia

La Brecha Operativa del Chatbot



1. Desacoplado del Sistema Operativo

El modelo responde texto pero carece de acceso a la terminal, el árbol de archivos o los comandos de compilación de Flutter.



2. Fricción Manual Continua

Copiar fragmentos, pegarlos en el editor, ejecutar `flutter analyze`, leer errores y redactar nuevos prompts manualmente.

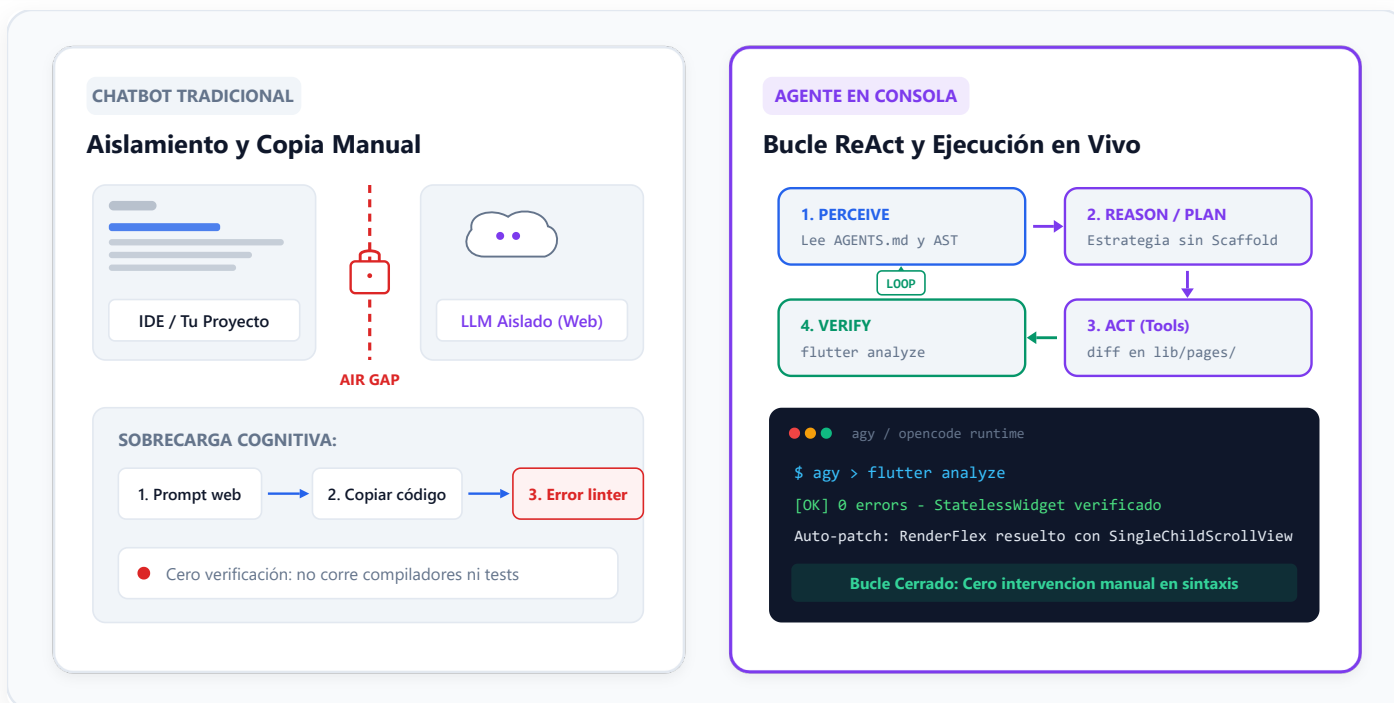


3. Amnesia Arquitectónica

Al no poder leer el `pubspec.yaml` ni las convenciones locales, el chatbot asume versiones obsoletas o inventa widgets.

El Agente en Consola y el Bucle ReAct

Bucle ReAct (Reason + Act): El agente en terminal ejecuta herramientas nativas (inspección de archivos, ejecución de shell y propuesta de diffs) observando el resultado antes de generar la respuesta final.



Anatomía Operativa del Bucle ReAct

1. THOUGHT (RAZONAMIENTO)

El agente evalúa el estado del repositorio y planifica el paso a seguir:

- ➔ Formula hipótesis técnica
- ➔ Elige la herramienta adecuada

2. ACTION (TOOL CALL)

Invoca una herramienta con argumentos estructurados JSON:

- 📄 Lectura de `pubspec.yaml`
- 📄 Búsqueda de clases en Dart

3. OBSERVATION (RETORNO)

El entorno devuelve la salida real generada por el sistema:

- ✓ Contenido exacto del archivo
- ✗ Salida stdout y stderr

4. SYNTHESIS (RESPUESTA)

Determina si completó la tarea o requiere otro ciclo:

- 📄 Presenta diff con cambios
- ✓ Solicita confirmación humana

Human-in-the-Loop vs Human-in-the-Lead

De la Aprobación Ciega al Liderazgo Técnico: Human-in-the-Loop reacciona pulsando Enter sin verificar; Human-in-the-Lead define el contrato previo, audita el diff propuesto y exige validación estricta de lints.



Pilares del Rol Human-in-the-Lead

1. AUDITORÍA ACTIVA DE DIFFS

Ninguna modificación se aplica a disco sin inspección minuciosa:

- ✓ Verificar que no rompa inmutabilidad
- ✓ Comprobar constructores `const`
- ✓ Rechazar dependencias innecesarias

2. GOBERNANZA ARQUITECTÓNICA

Imponer las reglas de diseño antes de generar cualquier línea:

- ⚙ Convención estricta Screen vs Page
- ⚙ 100% de la UI en StatelessWidget
- ⚙ Paleta Google Material Design 3

3. BARRERA DE CALIDAD POR LINTS

El código del agente debe someterse al analizador del lenguaje:

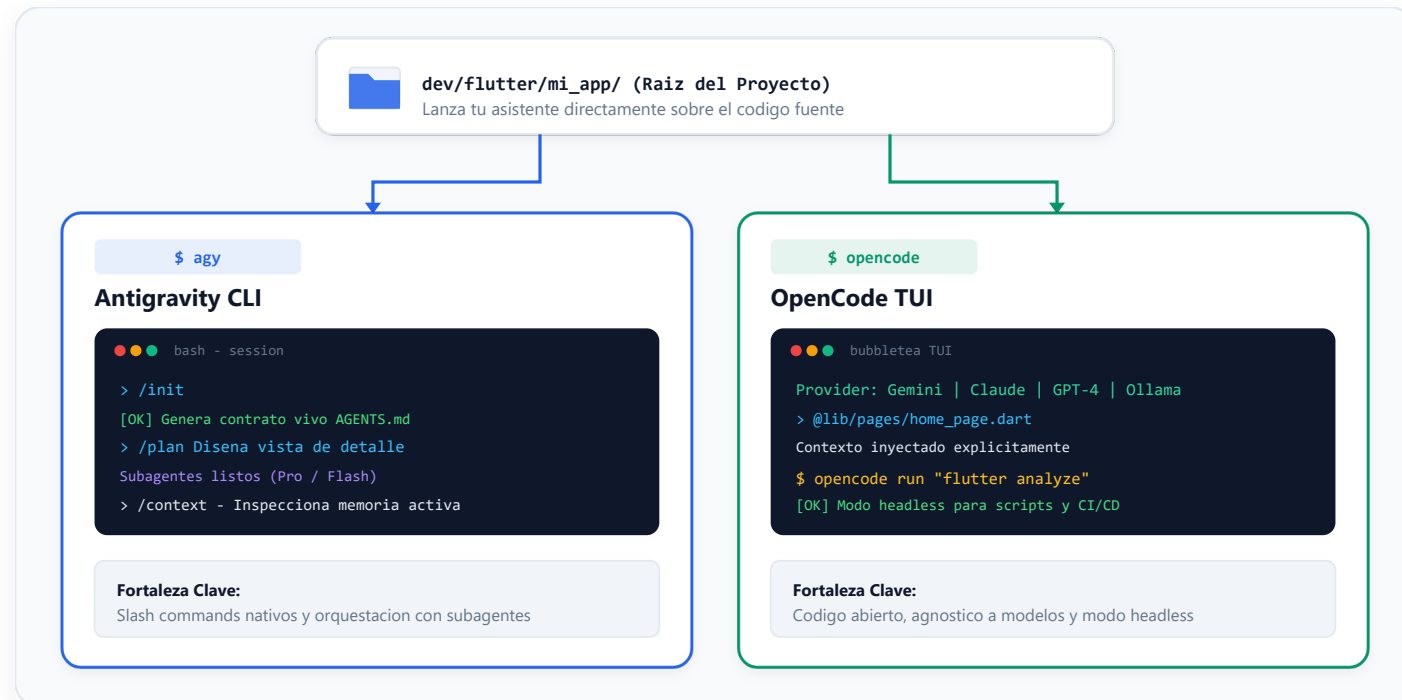
- 📄 Ejecutar `flutter analyze`
- ✓ Exigir 0 issues en la terminal
- ✓ Formateo con `dart format`

03. Herramientas CLI Agénticas

Ecosistema de clientes de terminal para interactuar con modelos de frontera directamente sobre el repositorio local.

Ecosistema CLI: Antigravity y OpenCode

Clientes de Terminal: Herramientas modernas que conectan el árbol de código con modelos de IA mediante ejecución de comandos, lectura de archivos y propuestas de parches en consola.



Comparativa Técnica: agy frente a opencode

Dos Filosofías de Terminal: Antigravity se especializa en orquestación profunda de agentes y gestión modular de contexto; OpenCode prioriza una interfaz interactiva TUI ligera con soporte de múltiples proveedores.

Criterio Técnico	Antigravity CLI <code>agy</code>	OpenCode CLI <code>opencode</code>
Propósito Principal	Orquestación agéntica avanzada con soporte de subagentes nativos.	Terminal User Interface (TUI) ligera en Go con multiproveedor.
Interfaz y Operación	Consola interactiva con Slash Commands (<code>/init</code> , <code>/context</code> , <code>/compact</code>).	Navegación interactiva por paneles y shortcuts de terminal (Bubble Tea).
Ecosistema de Modelos	Familia Gemini (Flash y Pro) con ventana extendida y herramientas nativas.	Gemini, Claude 3.5 Sonnet, GPT-4o y modelos locales vía Ollama.
Diferencial Clave	Detección nativa de reglas de arquitectura en <code>AGENTS.md</code> y skills.	Cliente 100% open-source, portable y sin dependencias de runtime.

¿Qué son los Comandos Slash (/) en Consola?

Directivas Locales de Control: Instrucciones especiales que inician con `/` (ej. `/context` , `/compact` , `/cost`). Son **interceptadas inmediatamente por el runtime local del CLI** para gobernar la sesión con **0 tokens de inferencia** .

Slash Commands: Directivas Locales vs Consultas al Modelo

Cómo el CLI intercepta comandos especiales que se ejecutan en tu máquina sin consumir tokens de inferencia

Terminal Prompt

CASO A: PROMPT NORMAL

> "Crea la pantalla de perfil"

CASO B: SLASH COMMAND (/)

> `/context` , > `/compact`

Viaja al LLM

Interceptado Local

Inferencia en el Modelo (Cloud)

- El prompt se añade al historial de contexto.
- Consume tokens de entrada y genera tokens de salida (costo USD y latencia).

Motor Local del CLI (0 Tokens de Inferencia)

- Lee la memoria local e imprime el peso de tokens en consola.
- Ejecuta compresión local y poda el historial de turnos.
- Configura el runtime sin enviar nada al modelo.

Slash Commands: Son el **panel de control del desarrollador**. Permiten gobernar la sesión, auditar la memoria y calibrar costos en tiempo real.

Protocolo de Inicio en Proyectos Flutter

1. DIAGNÓSTICO DE SALUD DEL ENTORNO

Antes de invocar agentes, el entorno debe estar completamente validado:

```
# Comprobar estado del SDK y herramientas  
flutter doctor -v  
  
# Lanzar emulador o conectar dispositivo físico  
flutter devices
```

Evita que el agente diagnostique falsos errores derivados de un SDK no configurado.

2. POSICIONAMIENTO Y DESCUBRIMIENTO

El agente debe posicionarse en la raíz del repositorio:

```
# Posicionarse en la carpeta raíz del proyecto  
cd mi_app_flutter  
  
# Lanzar el cliente agéntico en el repositorio  
agy # o bien: opencode
```

Permite al CLI cargar automáticamente el archivo `AGENTS.md` y el `pubspec.yaml`.

04. Gestión de Contexto

El modelo mental de la memoria del modelo: cómo se ensambla, por qué se satura y cómo auditarla en consola.

¿Qué es la Ventana de Contexto?

La Memoria a Corto Plazo: Es el búfer de tokens finito que se ensambla en cada interacción para alimentar la función matemática del modelo. Todo lo que el LLM comprende y evalúa en un turno debe coexistir dentro de este límite.

La Ventana de Contexto: La Memoria a Corto Plazo del LLM

El búfer de tokens finito que se reensambla en cada turno para alimentar la función matemática del modelo

Tu Repositorio

- Millones de líneas de código
- Cientos de archivos en lib/
- Historiales de git pasados
- Dependencias de Flutter

NO CABE TODO A LA VEZ
El LLM no puede "leer"
todo el disco en memoria.

Carga

Ventana de Contexto (Tokens)

1. System Prompt del CLI
2. Contrato AGENTS.md (Reglas)
3. Historial de Turnos Recientes
4. Herramientas y Estado Actual
5. Prompt Actual del Usuario →

Inferencia

Función LLM $f(x)$

- Sin Estado (Stateless)**
- Evalúa los tokens del búfer
 - Devuelve la respuesta
 - Olvida todo al terminar

RESPUESTA PRECISA
Depende 100% de lo
que esté en el búfer.

Regla de Oro: La ventana de contexto es la **única realidad del modelo**. Lo que no esté ensamblado en sus tokens en ese milisegundo, no existe.

El Mito de la Memoria vs la Ventana de Tokens

✗ El Mito Habitual

Creer que el LLM tiene "memoria continua", recuerda lo conversado ayer y retiene todo el repositorio en un cerebro persistente.

Consecuencia: Prompts ambiguos ("arregla lo de antes"), pérdida de arquitectura y frustración por respuestas desalineadas.

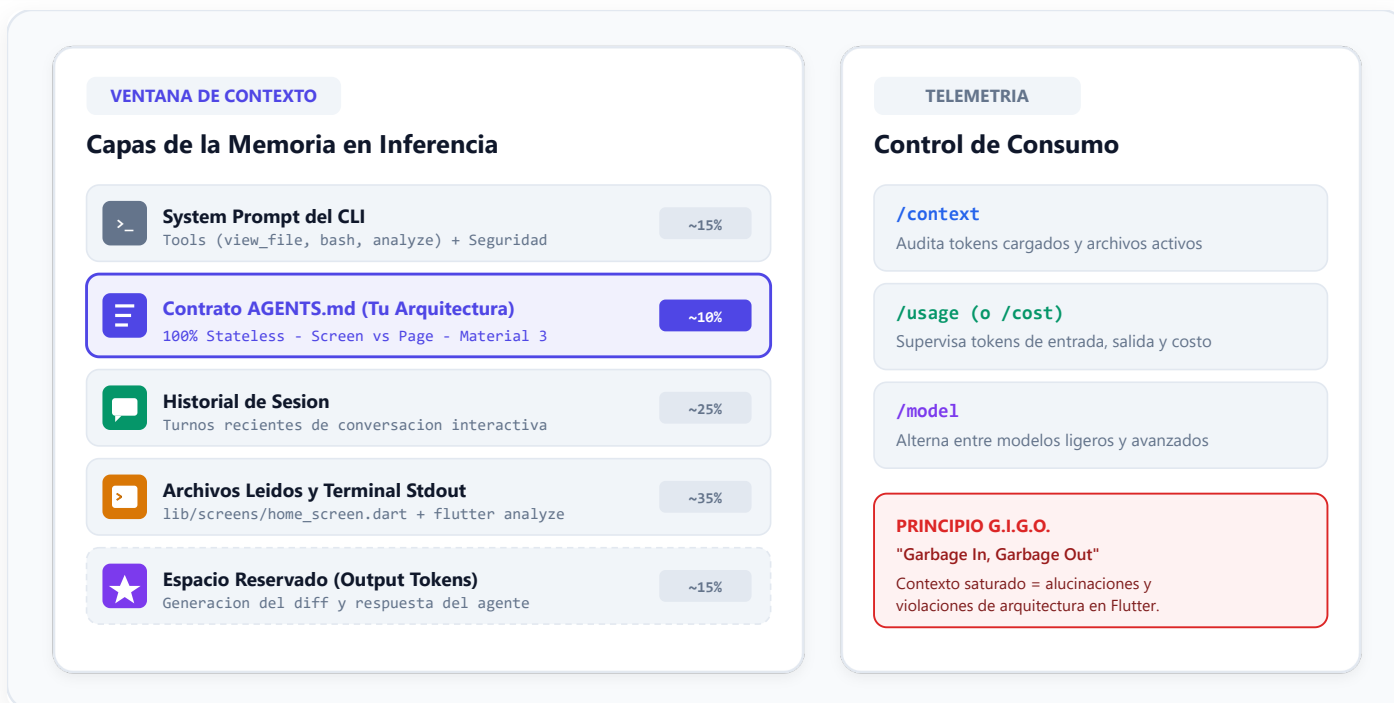
✓ La Realidad Matemática

Los LLMs son funciones puras sin estado: $f(\text{contexto}) \rightarrow \text{respuesta}$. Cada interacción reconstruye una ventana finita de tokens.

Principio de Ingeniería: Lo que no esté en la ventana de contexto al momento de la inferencia, simplemente **no existe** para el modelo.

Anatomía de la Ventana de Contexto

Estructura por Capas: Cada solicitud al agente ensambla el System Prompt del CLI, el Contrato AGENTS.md, el Historial reciente de la sesión, las Herramientas del sistema y el Prompt actual del desarrollador.



Comandos de Auditoría e Higiene en Terminal

/CONTEXT — AUDITORÍA

Inspecciona qué archivos y tokens están cargados activamente en la ventana.

-  Desglose por capa
-  Detección de archivos pesados

/COMPACT — COMPRESIÓN

Resume el historial de turnos pasados en un resumen ejecutivo ligero.

-  Libera hasta el 70% de tokens
-  Preserva decisiones clave

/COST — MONITOREO

Reporta el gasto acumulado en USD y consumo de tokens de entrada/salida.

-  Métricas en tiempo real
-  Control presupuestal

/MODEL — SELECTOR

Permite conmutar dinámicamente el motor de razonamiento agéntico.

-  Flash: Tareas rápidas
-  Pro: Arquitectura compleja



Precisión



Modular



Higiene

Reglas de Oro para Gobernar el Contexto



1. Brevedad Quirúrgica

Indica siempre la ruta exacta del archivo (ej. `lib/pages/home_page.dart`) y el widget específico a intervenir. Evita descripciones vagas.



2. Referencias a Memoria Modular

Enlaza documentos de convención (`.agents/memory/`) en lugar de duplicar instrucciones largas en cada prompt interactivo.



3. Poda Periódica con `/compact`

Comprime la sesión tras cerrar un ciclo de trabajo o reinicia el CLI al cambiar radicalmente de funcionalidad en la aplicación.

05. Contrato AGENTS.md

Cómo formalizar la arquitectura de software en un archivo ejecutable que la máquina interpreta con prioridad máxima.

Documentación Humana vs Contrato Agéntico

Manuales para Personas (README.md)

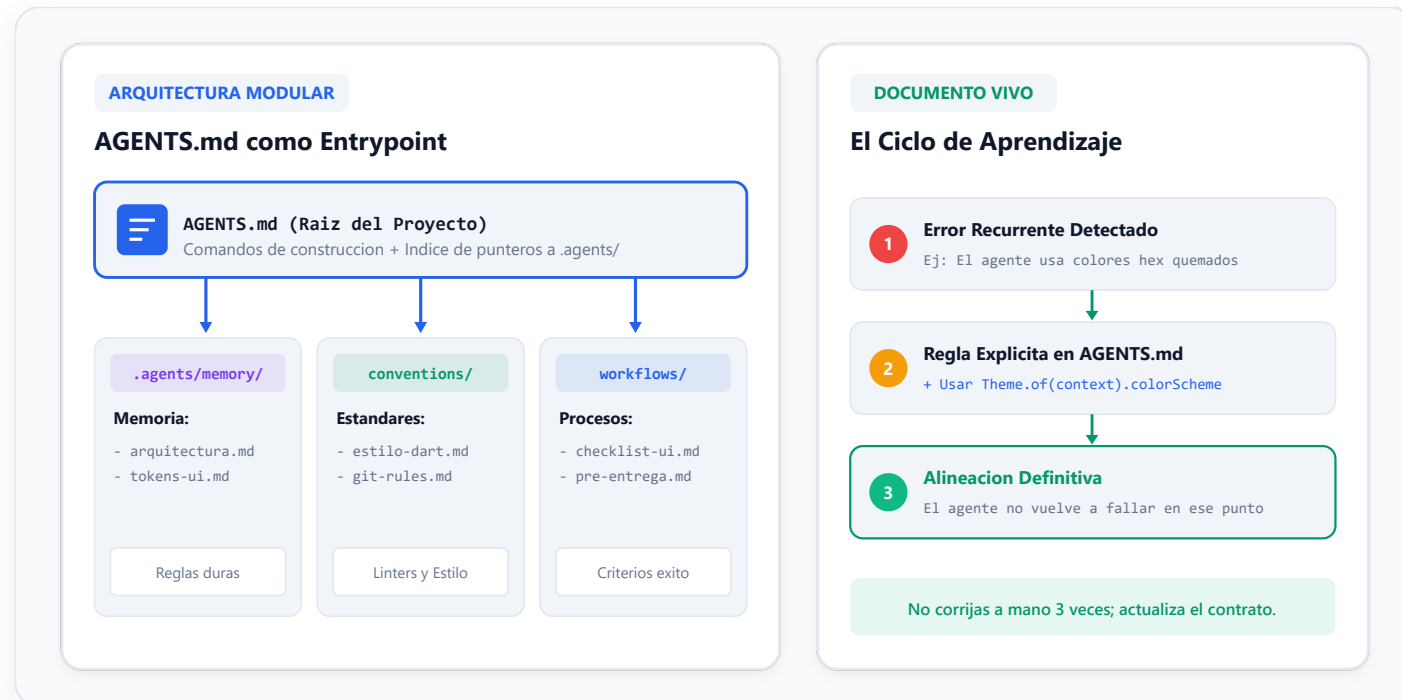
- ➔ **Objetivo:** Enseñanza pedagógica y bienvenida.
- ➔ **Estilo:** Prosa narrativa, analogías y contexto.
- ✗ **Problema para IA:** Diluye reglas técnicas en párrafos largos de cortesía y contexto histórico.

Contratos para Agentes (AGENTS.md)

- ✓ **Objetivo:** Restringir grados de libertad operativa.
- ✓ **Estilo:** Directivas imperativas, listas y rutas.
- ✓ **Verificación:** Comandos exactos(`flutter analyze`).

Estructura Modular de AGENTS.md

El Patrón Entrypoint: El archivo raíz `AGENTS.md` actúa como un índice ligero de alta prioridad que enlaza subdirectorios especializados dentro de `.agents/` para no saturar la ventana inicial del agente.



Distribución de Responsabilidades en .agents/

.AGENTS/MEMORY/ — MEMORIA

Preserva decisiones técnicas y reglas arquitectónicas permanentes:

- ✓ Regla estricta `Screen` vs `Page`
- ✓ Convención 100% `StatelessWidget`
- ✓ Historial de refactorizaciones

.AGENTS/CONVENTIONS/ — ESTILO

Estandariza sintaxis y diseño visual de la interfaz:

- ⚙ Paleta Google Material Design 3
- ⚙ Reglas de nomenclatura de carpetas
- ⚙ Sintaxis de constructores requeridos

.AGENTS/WORKFLOWS/ — PROCESOS

Secuencias ordenadas para tareas complejas recurrentes:

- 🔧 Protocolo de ejecución de pruebas
- 🔧 Limpieza de lints y formateo
- 🔧 Preparación de release móvil

El Ciclo del Documento Vivo (Iterative Rules Loop)

Ciclo de Mejora Continua: En lugar de corregir al agente manualmente una y otra vez, cada inconsistencia detectada se convierte en una directriz formal que evoluciona el contrato y previene fallos a futuro.

1. DETECTAR PATRÓN

El agente incurre en un error o sintaxis obsoleta (ej. usar `primary:` en lugar de `backgroundColor:` en botones Material 3).

2. ACTUALIZAR REGLA

Se añade la directriz imperativa en `AGENTS.md` o en `.agents/conventions/` señalando explícitamente lo **PROHIBIDO** frente a lo **REQUERIDO**.

3. RECARGA Y REFACTOR

El agente asimila la directriz sin reiniciar sesión (recarga con `/context`) y refactoriza el código de inmediato para ajustarse a la nueva regla.

4. COMMIT CONJUNTO

Se realiza un commit en Git que consolida simultáneamente el código fuente corregido y el archivo de contrato, blindando al repositorio.